Cache Controller with Snoopy Coherence — Design

Structure of Computer Systems · Project documentation, milestone M2 (Design)

Student: Arthur Kantor

Group: 30232

Coordinating teacher: Mădălin Neagu

Academic year: 2026–2027

Technical University of Cluj-Napoca

Faculty of Automation and Computer Science, Computer Science Department

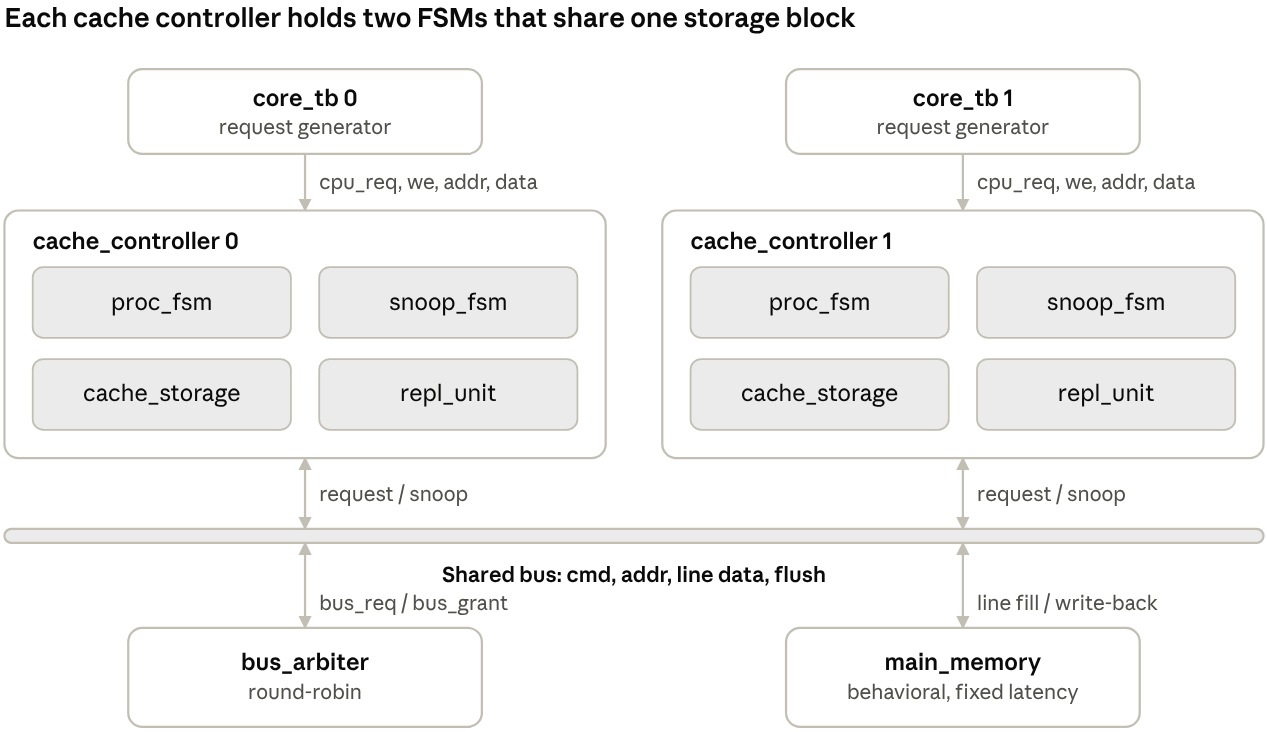
Table of Contents

- **1. Architecture and Components**
  - 1.1 System Architecture
  - 1.2 Entities
  - 1.3 Interface Signals
- **2. Bus Protocol**
- **3. Processor-Side FSM**
- **4. Snoop FSM**

1. Architecture and Components

The system is one top-level entity, dual_core_system, that instantiates two cores, two identical cache controllers, a bus arbiter and a main memory. Each cache controller is split into a processor-side FSM and a snoop FSM that share the same storage.

1.1 System Architecture



System architecture · entities and their connections

The cores talk only to their own controller. Everything else (misses, upgrades, write-backs) goes through the shared bus, which the arbiter gives to one controller at a time.

1.2 Entities

Entity	Instances	Role
dual_core_system	1	Top level; connects all components and the shared bus
core_tb	2	Testbench process issuing read/write sequences and checking returned values
cache_controller	2	Wraps proc_fsm, snoop_fsm, cache_storage and repl_unit
proc_fsm	1 per controller	Serves the core: lookup, hit/miss, write-back, bus requests, line fill

Entity	Instances	Role
snoop_fsm	1 per controller	Watches the bus and applies MSI transitions to the local copy
cache_storage	1 per controller	Tag, data and MSI state arrays, indexed by set and way
repl_unit	1 per controller	LRU ages or FIFO pointers; chooses the victim way
bus_arbiter	1	Round-robin grant between the two controllers
main_memory	1	Behavioral memory with a fixed read/write latency

1.3 Interface Signals

Generics (same for both controllers)

Generic	Example value	Meaning
NUM_SETS	64	Number of sets S
ASSOC	4	Ways per set N
LINE_SIZE	16	Bytes per line L
REPL_POLICY	LRU	LRU or FIFO
ADDR_WIDTH	32	Address width in bits
MEM_LATENCY	10	Memory access time in clock cycles

Ports of cache_controller

Signal	Dir	Width	Meaning
cpu_req	in	1	Core issues a request
cpu_we	in	1	1 = write, 0 = read
cpu_addr	in	32	Byte address
cpu_wdata	in	32	Write data
cpu_rdata	out	32	Read data

Signal	Dir	Width	Meaning
cpu_ready	out	1	Request completed
bus_req	out	1	Request for the bus
bus_grant	in	1	Bus granted by the arbiter
bus_cmd	inout	3	IDLE, BUSRD, BUSRDX, BUSUPGR, FLUSH
bus_addr	inout	32	Line address of the transaction
bus_data	inout	8 × LINE_SIZE	Whole cache line
snoop_flush	out	1	Other cache supplies the line (it held it in M)
bus_done	in	1	Transaction finished, data valid

2. Bus Protocol

The bus is atomic: once a controller is granted the bus, its transaction finishes before the arbiter grants it again. One transaction runs in five steps:

1. The requesting controller asserts bus_req.
2. The arbiter asserts bus_grant for one controller (round-robin when both request).
3. The granted controller drives bus_cmd and bus_addr (and bus_data for FLUSH).
4. The other controller's snoop_fsm looks up bus_addr in its own cache:
   - line in M and BUSRD or BUSRDX: it asserts snoop_flush and drives the line on bus_data;
   - line in S and BUSRDX or BUSUPGR: it invalidates its copy;
   - otherwise it does nothing.
5. Memory finishes the transfer and asserts bus_done:
   - with snoop_flush, memory writes the flushed line and the requester takes the same data from the bus (cache-to-cache transfer);
   - without snoop_flush, memory supplies the line after MEM_LATENCY cycles;
   - BUSUPGR carries no data and completes in one cycle.

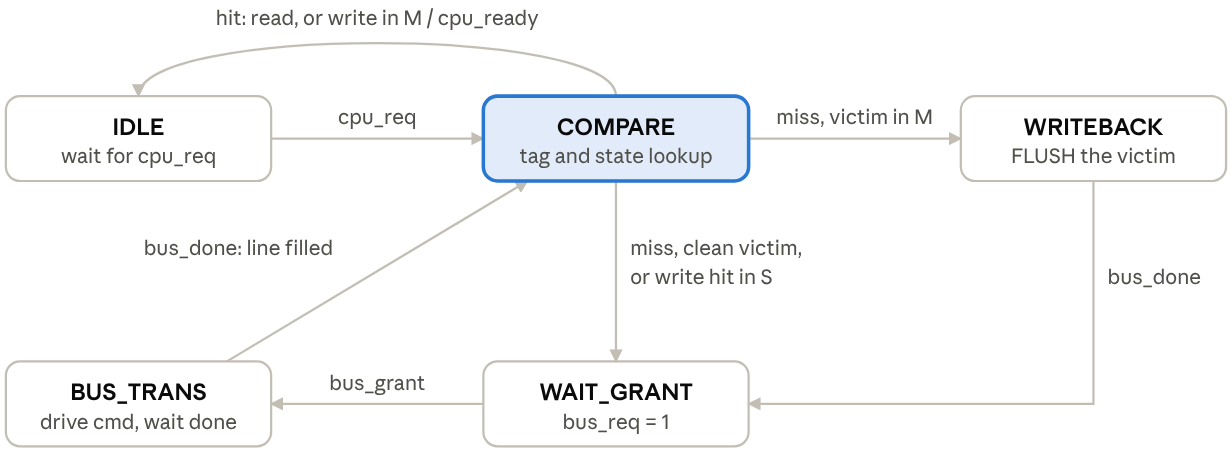
Evicting a line in M is its own FLUSH transaction, done before the request for the new line.

Upgrade race. If both caches hold a line in S and both write it, both request BUSUPGR. The winner invalidates the loser's copy, so the loser's proc_fsm changes its pending command from BUSUPGR to BUSRDX before it gets the grant.

3. Processor-Side FSM

proc_fsm serves one request from its core at a time. After a miss it fills the line and returns to COMPARE, where the access now hits, so reads and writes are always completed in the same place.

Every miss ends with a line fill and a second lookup that hits



proc_fsm · 5 states

State	Actions	Next state
IDLE	cpu_ready = 0; latch cpu_addr, cpu_we, cpu_wdata	COMPARE on cpu_req
COMPARE	Split address into tag, index, offset; compare the N tags of the set	Read hit (S or M) or write hit in M: return or write the word, update repl_unit, line becomes M on write, cpu_ready = 1, go to IDLE. Write hit in S: command BUSUPGR, go to WAIT_GRANT. Miss: repl_unit picks the victim way; victim in M: go to WRITEBACK; else command BUSRD (read) or BUSRDX (write), go to WAIT_GRANT

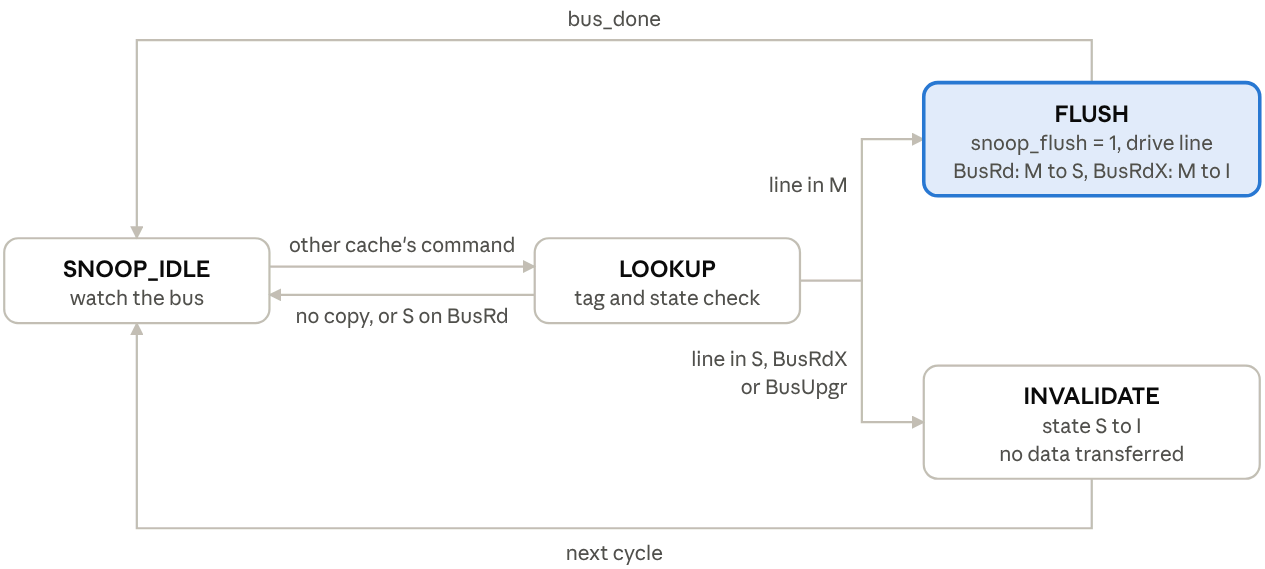
State	Actions	Next state
WRITEBACK	Request the bus, issue FLUSH with the victim line; victim state becomes I	WAIT_GRANT on bus_done
WAIT_GRANT	bus_req = 1; if the line was invalidated while waiting, BUSUPGR becomes BUSRDX	BUS_TRANS on bus_grant
BUS_TRANS	Drive bus_cmd and bus_addr; on bus_done store the line and tag, set state S (BUSRD) or M (BUSRDX, BUSUPGR)	COMPARE on bus_done

When proc_fsm and snoop_fsm need cache_storage in the same cycle, snoop_fsm has priority because the bus transaction is waiting for its answer; proc_fsm stalls for that cycle.

4. Snoop FSM

snoop_fsm reacts only to transactions issued by the other controller; it ignores its own controller's commands and the FLUSH of an evicted line.

Only a line held in M makes the snooping cache drive the bus



snoop_fsm · 4 states

Snooped command	Local state	Action	New state
BUSRD	M	Flush the line (memory and requester take it)	S
BUSRD	S or I	None	Unchanged
BUSRDX	M	Flush the line	I
BUSRDX	S	Invalidate	I
BUSUPGR	S	Invalidate	I
Any	I	None	I

A BUSUPGR can never find the line in M in the other cache, because the requester holds it in S, so the other copy is at most S.